SIGNAL HARBOR

Technical Specification

The system as built: architecture, components, data, state and modes, security, deployment, APIs, testing, limits. Prepared for the client showcase, 12 October 2026. Demo company: Evergreen Office (synthetic data only).

Version for the client showcase, 12 October 2026. Audience: the CTO, head of data and engineers. Status words used throughout: **Built** means in the code and covered by tests. **Planned** means not built. The demo company is Evergreen Office, a synthetic family office. No real client data exists in the product, the tests or this document.

1. Purpose and scope

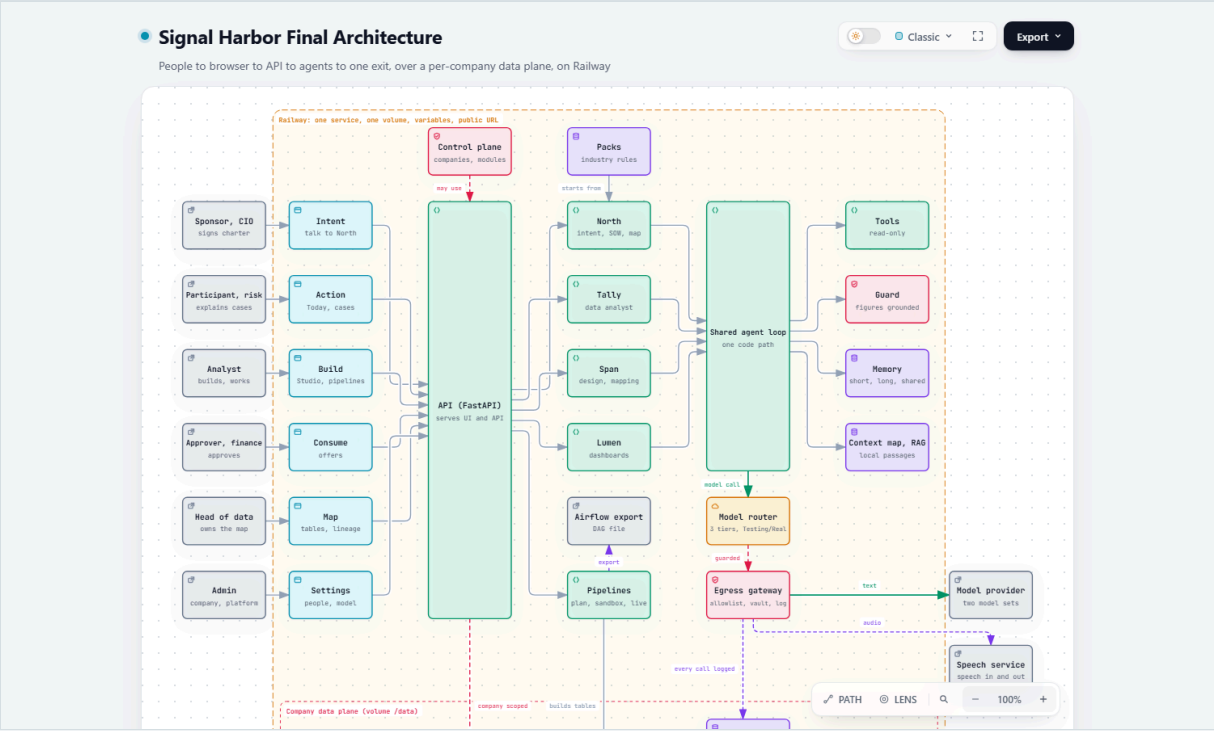
Signal Harbor takes a business from "what do we want to know" to "what is likely next". An agent drafts at every step; a named person signs every result. This specification describes the system as built: what runs, where data lives, what can leave the machine, how it is deployed and how it is tested.

In scope: the web application, its four centres, the four agents, the model router, the egress gateway, the per-company data plane, state and job handling in the two run modes, security, deployment on Railway, the API families, and test results. Out of scope: user guides (see the manual) and commercial terms.

Design rules that shape everything below:

1. Real client data never leaves the client's environment. Tests and demos use synthetic data only.
2. An agent has exactly one way out: the egress gateway.
3. Contracts and specs are the source of truth; no table, column or threshold is invented in code.
4. Signed content is read-only; a draft never overwrites a human edit.
5. The industry is a parameter, never hard-coded in a module. Industry packs hold suggestions, not company policy.

2. Architecture overview



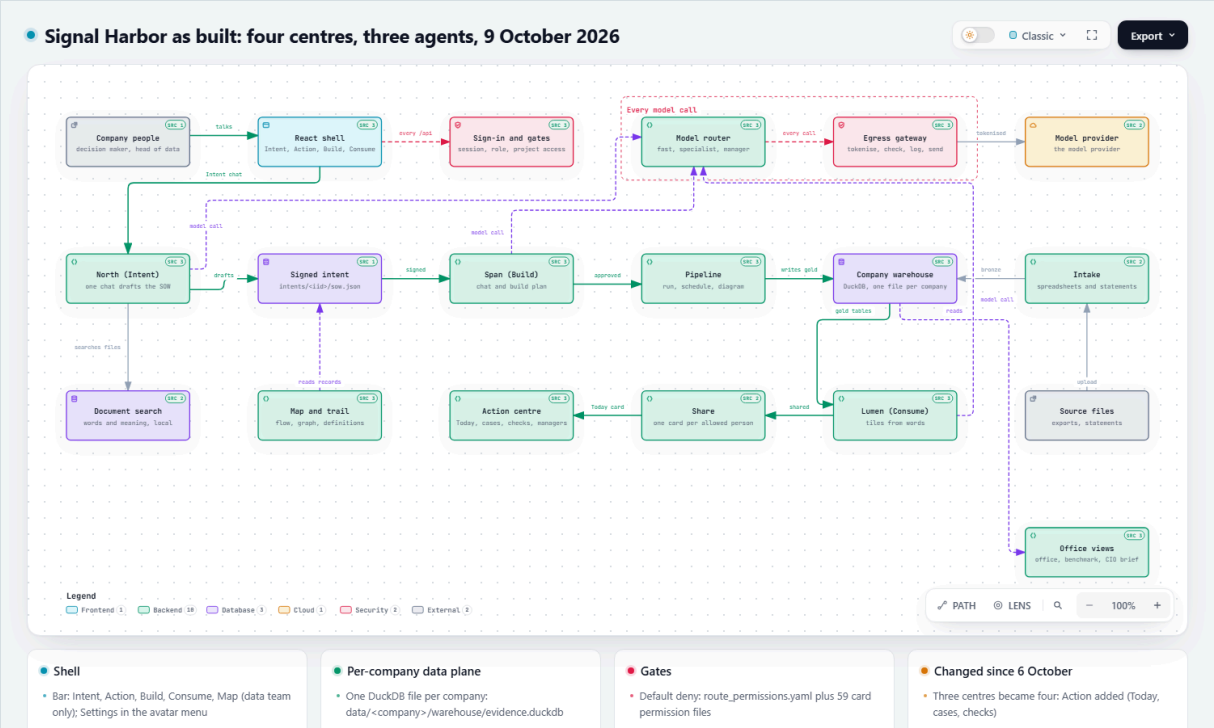
Final architecture (Archify diagram: sh-final-architecture)

2.1 Three planes

Plane	Holds	Does not hold
Control plane	Super admin, module switches, industry packs, domain context	Any company data
Module plane	The modules and their agents; the same code for every company	State of its own
Data plane (one per company)	Landing zone, warehouse, intents, baseline, context, graph, logs	Anything from another company

Every path, table and graph is resolved from the signed-in company. A request that would read another company's plane fails closed.

2.2 The four centres



Four centres (Archify diagram: sh-four-centres)

Centre	Purpose	Main screens
Intent	Talk to North; the signed intent of work and process map come out of one conversation	Chat, intent, process map
Action	The person's day: cases, reconciliations, statements to chase, figures to confirm, approvals; Ask Tally	Today (one page), cases, tickets
Build	Design and run the data pipeline	Projects, Architecture (design pack), Pipelines, Dashboard studio, Operations
Consume	Published figures for the readers	Today (Morning Note), Financial health, Markets, Reports, What if

Beside the centres: **Map** (the office's ontology, process map and business glossary; tables and lineage) and **Settings** (people, connectors, jobs and events).

2.3 The four agents

Agent	Role	Where
North	Drafts the intent of work (SOW) and the process map live from a conversation; asks how each named system arrives and how often	Intent
Span	Two roles. **Architect**: reads the signed intent, asks only what is missing, produces the design pack (target model, catalogue, ETL mapping sheet, rules, sources); the head of data approves. **Builder**: connects, builds and runs from the mapping sheet on a copy, asks for a benchmark, shows variance, goes live on approval; hands a variance back to the architect with the evidence	Build
Lumen	Proposes a starter dashboard from the intent's KPIs, readers and decisions; writes briefs and the Morning Note; a different person approves before it shows in Consume	Build, Consume

Agent	Role	Where
Tally	Data analyst of the Action centre: explains cases, finds timing causes, drafts follow-ups (never sent)	Action

All four run on one shared, read-only agent loop (section 3.3). Agents propose; people act. Humans approve at defined points: the intent, the design pack and build plan (head of data), go-live, dashboards (a different person from the author) and cases.

The **never-ask-twice** rule: whatever the intent captured about a system (what it holds, owner, format, how it arrives, how often) is carried forward and shown for confirmation, never re-entered. A mismatch is raised once, with the intent's value beside it.

3. Components

3.1 Application server (FastAPI)

One Python 3.11 process (uvicorn, app factory `sh_platform.app:create_app`) serves the API and the built UI. Routes are discovered from the modules and from `sh_platform/v1`; each route group ships a `*.permissions.yaml` file (75 of them in `sh_platform/v1`) beside the central `route_permissions.yaml`. A route that is not declared is refused (default deny). Built.

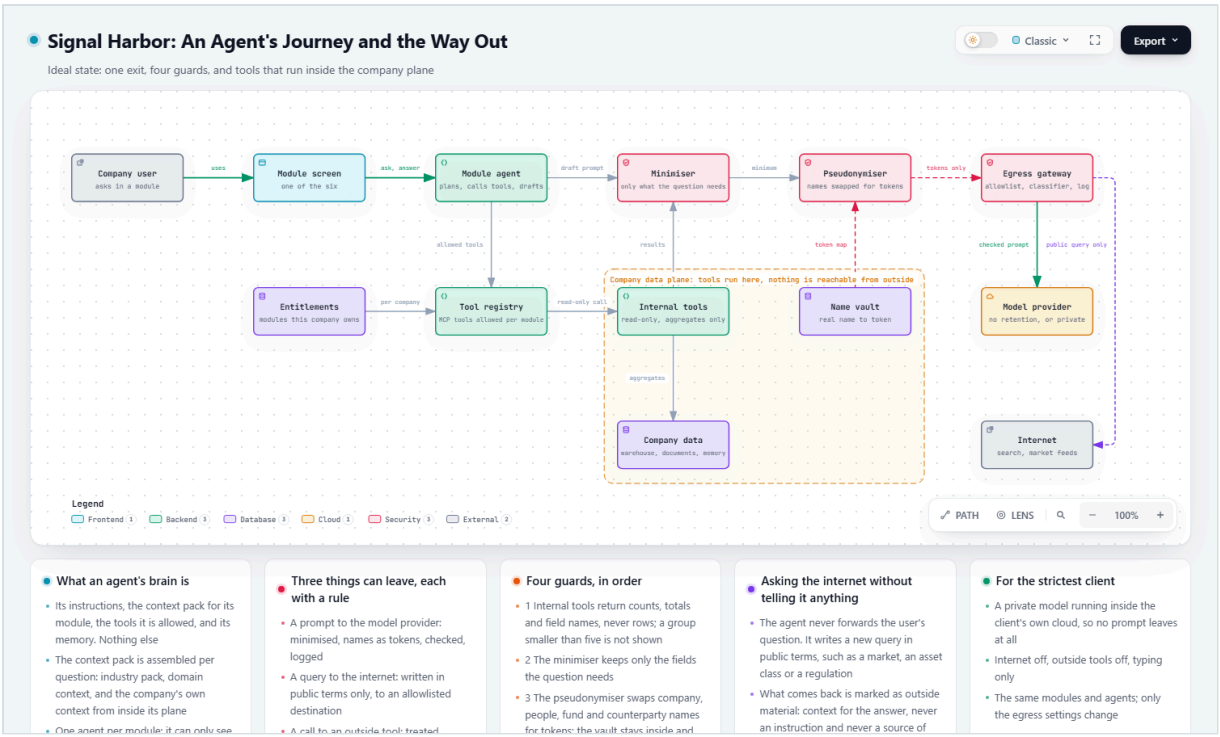
3.2 Browser UI

React and TypeScript, built with Vite in the first Docker stage and served as static files by the same process. Screens follow approved artboards and are checked by screenshot against them. Built.

3.3 Agent runtime

- One loop for all agents. Tools are read-only and return counts, totals and field names, never rows. A group smaller than five is not shown (under-five rule).
- A **reply guard** checks every figure in an answer against the facts the tools recorded; a figure that cannot be traced is replaced. Outside text (documents, web, outside tools) is data, never an instruction.
- Memory: short term (the turn), long term (per company) and shared; a context map with local meaning search over the company's documents (local embedding model, cached on the volume, no call out).
- Skills per agent are text files read at run time; handoffs between agents (Span architect and builder, Tally and Span) are recorded and visible to people.

3.4 Model router and tiers



Agent egress (Archify diagram: sh-agent-egress)

The router (`sh_platform/v1/router.py`) picks one of three tiers for each agent call by rules only (no model call to decide): **fast**, **specialist**, **manager**. The model provider is reached only through the egress gateway.

Each company has a mode, switchable without a restart, defined in `sh_platform/v1/model_modes.yaml` :

Mode	Purpose	Tier models
Testing	Cheap, tool-capable models for demos and tests	Set in the file for each tier; specialist and manager may be the same model
Real use	The production set	Set by environment variables per tier (`SH_MODEL_FAST` , `SH_MODEL_SPECIALIST` , `SH_MODEL_MANAGER`) and an alternate per tier (`SH_MODEL_<TIER>_ALT`)

Fallbacks, all logged as a line in the company's cost log:

1. **Retry up, never down.** An unusable reply (empty, not the asked JSON, or a figure the guard would rewrite) is asked once more on the next tier.
2. **Provider failure.** The call goes to the tier's alternate model in Real use; with none, to the next tier in Testing.
3. **Tier above unreachable.** The reply already in hand goes to the guard; a manager-first call that cannot be reached tries the specialist once (availability, never quality).
4. **Same model in both tiers.** No retry is made to a tier whose model in this mode is identical; the step is logged as "no stronger model in this mode".

Cost control: a per-company daily spend cap (`SH_DAILY_MODEL_CAP_USD` , read from the cost log; after it the agents answer from rules with a plain note) and a per-account chat turn limit per hour

(`SH_CHAT_TURNS_PER_HOUR`). `SH_ROUTER=off` pins every call to the specialist model. Prices per model are kept in `model_modes.yaml` and every call records `cost_usd`; unlisted models are priced at the tier default and marked estimated. Built.

3.5 Egress gateway

The gateway (`sh_platform/egress/`) is the only exit. Four guards, in order:

1. Internal tools return counts and totals only; groups under five are suppressed.
2. **Minimiser**: only the fields the question needs.
3. **Name vault** (pseudonymiser): company names, people's names, emails, phone numbers, fund, account and portfolio names, counterparties, account and policy numbers are swapped for tokens. The vault stays inside the company plane; replies are mapped back inside it.
4. **Allowlist and checks**: only listed destinations (hosts are fixed per provider; a company's internet switch is off by default and set per company and per module); any payload containing a vault name or a raw record is blocked; payload structure is checked against allowed keys; every call is logged (host, decision, masked count, size).

Never sent at all: raw rows; figures other than totals over groups of five or more; uploaded documents in full (only a short summary or the few relevant passages, tokenised, marked as untrusted quoted content). An **outage breaker** stops calls to a provider that is down (a dictionary in simple mode, Redis in production mode). Built.

3.6 Scheduler and job queue

A scheduler thread checks every minute and runs the plans whose schedule is due, live; a delivery loop sends due deliveries. Plans can be exported as an Airflow DAG. Work runs through a `JobQueue`: in simple mode jobs run inline on the spot (or on one background thread); in production mode a Redis queue holds ready jobs, a delayed set for retries with doubling backoff, and a failed list; a unique key stops duplicate waiting runs across replicas; a running job holds a renewable lease so a dead worker's job is retried. Handlers: pipeline tick, on-arrival run, pipeline run, delivery tick, graph rebuild. A Jobs and events page (Settings) shows the mode, failed jobs with "Send again", recent jobs and the live event stream. Built.

3.7 Connectors

Connectors (data engineer seat) bring files and data into the Landing zone. Three kinds: mailbox (IMAP), SFTP, and HTTPS. Specs hold environment variable **names**, never values; every outbound connection goes through the egress gate (allowlist and log). Each has Test, Dry run and Fetch, and a health record.

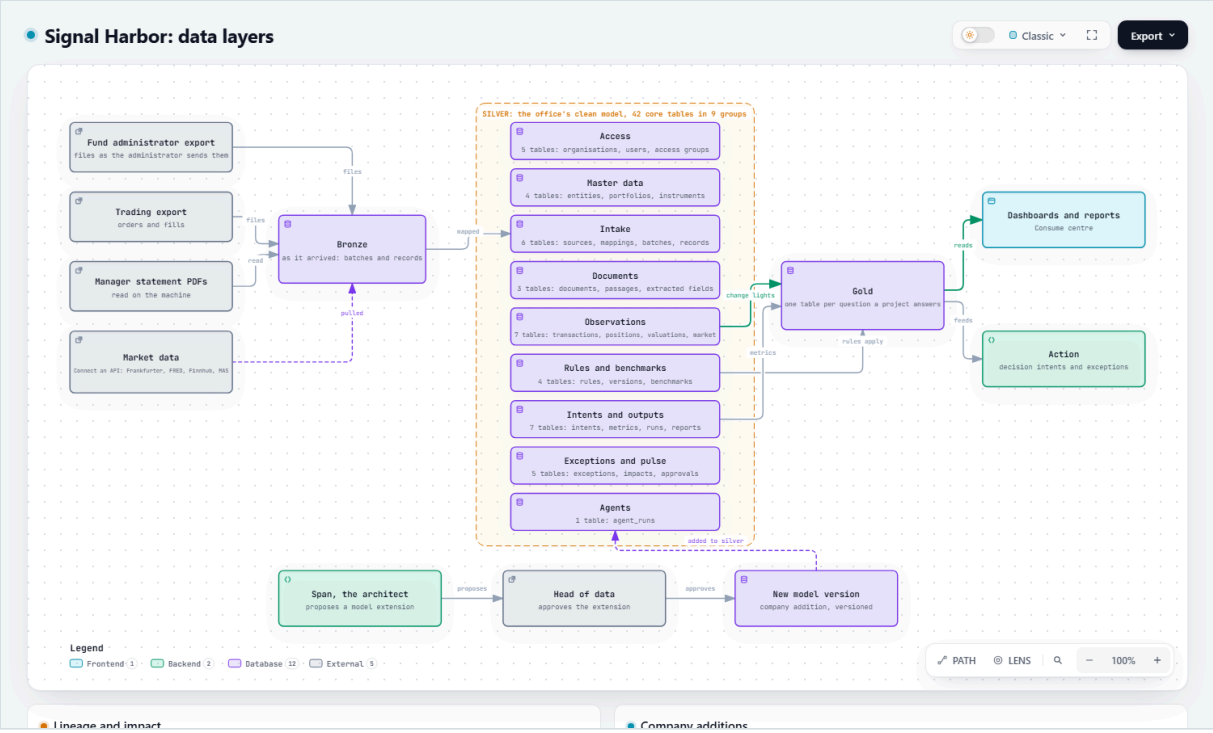
Connect an API (Build) is built on the same connectors, with a fixed provider list:

Provider	Host (fixed)	Key env name	Series offered
Frankfurter (ECB reference rates)	api.frankfurter.dev	none	USD/SGD, EUR/USD, EUR/SGD, GBP/USD
FRED	api.stlouisfed.org	`FRED_API_KEY`	DGS10, DGS2, DEXSIUS, Singapore CPI inflation

Provider	Host (fixed)	Key env name	Series offered
Finnhub	finnhub.io	`FINNHUB_API_KEY` (sent as a header, never in the address)	ACWI, AGG, EWS quotes; market news
MAS (Singapore Government Securities)	eservices.mas.gov.sg	none	SGS 2/5/10/15/20/30-year auction cut-off yield (last auction per tenor, not a daily curve)

Series are recorded as market observations with their source system and `connection_method = live_api`; Consume tags each outside line FILE or LIVE API from that record. A free-form "any REST API" option was left out on purpose. SORA is **planned**: it needs a free MAS API key (section 10). Built, except where marked.

4. Data architecture



Data layers (Archify diagram: sh-data-layers)

4.1 Warehouse per company

Each company has its own DuckDB file, `data/<company>/warehouse/evidence.duckdb`, plus its landing zone, documents, graph, logs and traces under `data/<company>/`. Nothing else reads or writes there except through the platform.

4.2 Bronze, silver, gold

Layer	Content	Rule
Landing	Originals as received (files, statements, exports)	Kept; never edited
Bronze	Loaded as is, prefix `bronze_`	No business meaning added

Layer	Content	Rule
Silver	The company model: the core tables, cleaned and typed	Changed only through model versions
Gold	Per project: reporting tables built from the signed mapping sheet, prefix `gold_`	Column types and definitions come from the design pack

For tools such as DBeaver, the schemas `bronze`, `silver` and `gold` hold **alias views** over the real tables in `main` (`silver.position_snapshots`, `gold.<x>` for `gold_<x>`, `bronze.<x>` for `bronze_<x>`). Real tables are never renamed; the aliasing is idempotent.

4.3 The core model: 42 tables in 9 groups

Source of truth is DBML (`docs/v2/evidence-to-action-mvp.dbml`).

Group	Tables
access (5)	organizations, users, access_groups, access_group_memberships, access_group_scopes
master_data (4)	legal_entities, portfolios, instruments, instrument_identifiers
intake (6)	source_systems, sheet_mappings, sheet_mapping_columns, source_batches, batch_control_totals, source_records
documents_and_rag (3)	documents, document_passages, extracted_fields
observations (7)	transactions, position_snapshots, valuation_observations, commitments, cash_movements, market_observations, nav_snapshots
rules_and_benchmarks (4)	business_rules, rule_versions, benchmark_definitions, benchmark_components
intents_and_outputs (7)	decision_intents, metric_definitions, calculation_runs, metric_values, metric_evidence, reports, report_versions
exceptions_and_pulse (5)	exceptions, exception_impacts, exception_events, approvals, pulse_items
agents (1)	agent_runs

4.4 Model versions and extensions

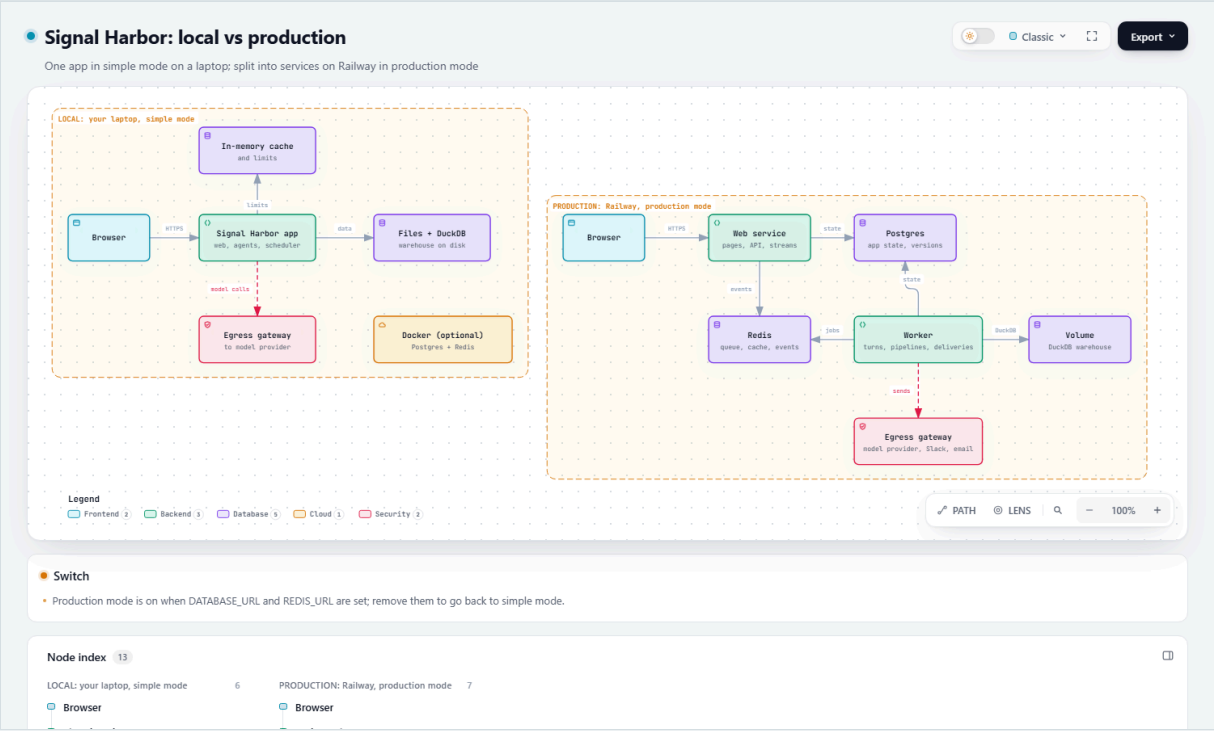
The data model is a governed catalogue (`sh_platform/v1/e0a_catalogue.py`). Version 1 is the core file. A company adds an **extension**: new tables, or new columns on core tables. It never drops or retypes anything core ("Core tables change only through a product release"). A proposal is a version in state `proposed`; **the head of data approves it** and it becomes current. Versions are kept per company (`model.dbml`, `extension.json`, `catalogue.json`, `meta.json`, a log). The warehouse follows by adding tables and columns only, with no constraints beyond primary keys and not-null. Extensions overlay by table name; a block naming a core table lists only the columns to add.

4.5 Lineage

Every figure is traced from Consume back through Gold, Silver, Bronze and Landing to the source file or API call, using the mapping sheet (source field, bronze, silver, gold, transformation or

formula), batch and source-record keys, and `metric_evidence`. The Map shows the trail and the graph (people, projects, sources, tables, dashboards, definitions, cases, decisions). The Morning Note and Markets show source, date and FILE or LIVE API for every outside figure.

5. State and modes



Local versus production (Archify diagram: sh-local-vs-production)

	Simple mode	Production mode
Chosen when	`DATABASE_URL` and `REDIS_URL` not both set	Both set. Only one set stops the server at start-up with a plain message
Company state	JSON files, atomic write, per-file lock; version is a hash of the file	`PgStore` on Postgres: one table `sh_state_<kind>` per store kind, JSONB document, bigint version, `put(expected=...)` compare-and-set raising `Conflict`; optional caller transaction
Jobs	Inline or one background thread	Redis queue plus a worker process (`tools/worker.py`)
Events	In-process bus, last 500 events	Postgres outbox: the event row is written in the caller's own transaction; `FOR UPDATE SKIP LOCKED` dispatch; each consumer runs once per event; a poison event stops after 20 attempts
Cache, rate limit, breaker	In process	Redis (answer cache with a data-version counter, sliding-window turn limit, outage breaker)
Warehouse (DuckDB), uploads, logs	Volume	Volume (by design, non-JSON data stays on disk)

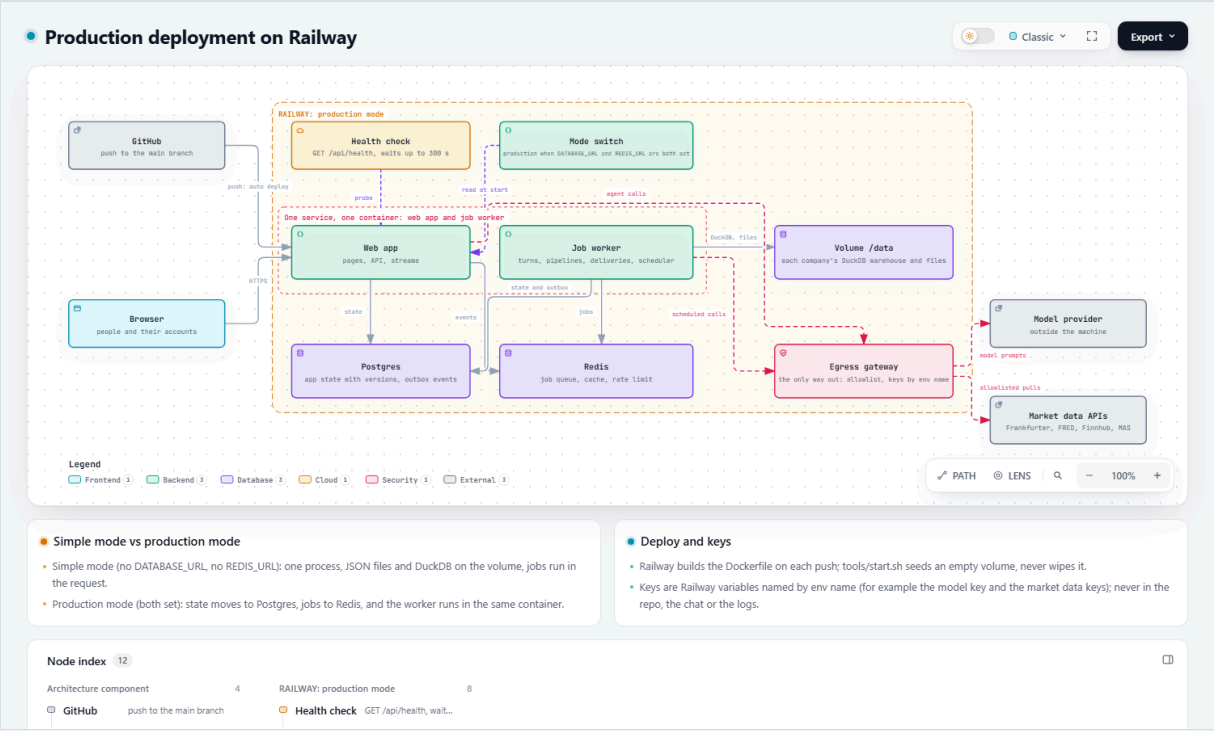
Behaviour is identical in both modes; the screens and routes do not change. The event stream to the browser is server-sent events (`GET /api/v1/events/stream`), filtered to the signed-in company; the screen reconnects about every 25 seconds.

Current position (honest note). The `StateStore` adapter, the Postgres schema, the migration (`tools/migrate_to_postgres.py`, idempotent, with `--dry-run`) and a parity checker (`tools/parity_check.py`) are built. Many existing stores (about 350 call sites) still read and write JSON files in both modes; in production mode the move to Postgres covers jobs, events, cache, limits and breaker, and the web service and worker share the data volume. Moving each store through the adapter is **planned**. Postgres connections are opened per call (no pool).

6. Security and data locality

- **Sign-in.** Email and password; passwords stored as scrypt hashes. Sessions: signed HttpOnly cookie plus a server-side session table; idle timeout 30 minutes, absolute 8 hours; login lockout after 5 failures for 60 seconds. Outside dev mode the server refuses to start without `SH_SESSION_SECRET` of at least 32 characters; secure cookies are on. No sign-up page.
- **CSRF.** A second cookie and an `x-csrf-token` header compared in constant time on state-changing requests; failure returns 403.
- **Roles and seats.** Roles: super_admin (control area only, no company data), company_admin, sponsor, analyst, head_of_data (held in addition to analyst), approver, participant, owner. Seats map roles to actions: decision maker, architect, data engineer, ops manager, head of data, consumer. A refused action returns plain words the screen shows.
- **Default-deny route permissions.** Every route is declared with `public`, `authenticated` or a role list; an undeclared route returns 403. Mounted apps are for static files only.
- **Egress rules.** Section 3.5 and the egress rule: one exit, tokenised names, no raw rows, allowlist, log, under-five rule, internet off by default per company.
- **Keys by environment name.** Specs, code, docs and the repo hold variable names only. Public demo: the super-admin routes are off and no super-admin is seeded; Real use mode is refused unless explicitly allowed.
- **Fund-level access.** Access groups with scopes (legal entity or portfolio, levels full or read) decide which structured rows a person sees; "preview" never opens rows; fail closed on any error; the company admin sees all; admin changes are logged. **Data-level restrictions beyond this (column and document-type) are planned** (section 10).
- **Four-eyes.** Dashboards are approved by a different person from the author; a cloud push needs two named approvers and, in a pilot, synthetic data only.
- **Security reviews done.** (1) Two-modes security review, 11 Oct: mode switch and start-up refusal, company scoping of the jobs page and event stream, permission files, Redis keys and TTLs, outbox once-only consumers, Postgres compare-and-set. Five defects fixed: event stream thread exhaustion, stream reconnect loop, stale event id, double retry, poison outbox event. (2) Production hardening (PM1): Redis password required, job leases for dead workers. (3) Pre-publish audit, 10 Oct: the commit set and image searched for secrets, personal data, other-project names and real names; none found; the working record is excluded from the repo. Known, accepted: the Redis rate window is count-then-add (a burst across replicas can pass the limit by a few turns); after the breaker window every caller may try a down provider once.

7. Deployment



Railway deployment (Archify diagram: sh-deployment-railway)

- **Docker image.** Stage 1 (Node 22) builds the UI. Stage 2 (Python 3.11 slim) installs `requirements.txt`, copies the platform, modules, packs, skills, seed and worker tools, the data model scripts and the synthetic universe, and the built UI. `/data` is linked to `/app/data`. Defaults baked in: `SH_MODE=prod`, `SH_DATA_ROOT=/data`, `SH_SCHEDULER=on`, `SH_ROUTER=on`.
- **Start script (`tools/start.sh`).** Seeds an empty volume (never wipes; generated logins go to `/data/_control/seed_credentials.txt`, never to the log). If `DATABASE_URL` and `REDIS_URL` are both set: migrates seeded data to Postgres (idempotent) and starts the worker in the background. Then `exec uvicorn ... --proxy-headers` on `$PORT` (default 8200).
- **Railway services.** Web and worker run **in one container**, because a Railway volume attaches to one service and the DuckDB warehouse must be shared. Plus a Postgres service and a Redis service. One volume mounted at `/data`. Do not run more than one replica of the web container.
- **Health check.** `GET /api/health`, timeout 300 s (first boot seeds the demo in about 30 s); restart on failure, up to 5 times (`railway.json`).
- **Environment variables, by name only.**

Name	Use
`SH_SESSION_SECRET`	Session signing (required, 32+ characters, set by hand)
`SH_DATA_ROOT`	Data folder (`/data`)
`SH_MODE`	`prod` turns on secure cookies and strict checks
`DATABASE_URL`, `REDIS_URL`	Both set turns on production mode
`SH_MODEL_PROVIDER`	Which provider the gateway uses for the agents

Name	Use
The model provider's key variable, and optional keys for speech and a second provider	Set as secrets in Railway; the agents need the first. Names are in the deployment guide
`FRED_API_KEY`, `FINNHUB_API_KEY`	Connect an API keys, only for those providers
`SH_MODEL_FAST`, `SH_MODEL_SPECIALIST`, `SH_MODEL_MANAGER`, `SH_MODEL_<TIER>_ALT`	Real use model per tier
`SH_DAILY_MODEL_CAP_USD`, `SH_CHAT_TURNS_PER_HOUR`	Spend cap and chat rate limit
`SH_ALLOW_REAL_MODE`, `SH_ENABLE_SUPERADMIN`	Off on any public link
`SH_SCHEDULER`, `SH_ROUTER`, `SH_PUBLIC_BASE_URL`, `SH_DEMO_PASSWORD`	Scheduler, router switch, public link base, optional shared demo password
`PORT`	Set by Railway

For the client's own environment the same image runs on the client's choice of host; the data stays on the client's volume or database.

8. APIs

All routes are under `/api`, JSON, cookie session plus CSRF header, default deny. Main families:

Family	Examples	Seat or role
Auth and profile	sign in, sign out, who am I, viewer profile	public / any signed-in
Health	`GET /api/health`	public
Intent (North)	chat turns, intent, process map, sign	sponsor, analyst, participants
Action	`GET /api/v1/action/landing`, `/cases`, `/recon/runs`, `/managers/inbox`, `/extract/queue`, tickets	analyst and consumer seats
Build	design pack, plans, plan approve, runs, benchmark, go-live, studio and dashboards	architect, data engineer, head of data
Connectors	`GET/PUT/DELETE /api/v1/connectors[/id]`, `POST .../test`, `.../dry-run`, `.../fetch`, `GET .../health`; Connect an API	data engineer
Consume	`GET /consume/today`, `/consume/health`, signals (with provenance), note, pnl, reports	consumer
Map and graph	graph, process map, glossary, trail, review queue	analyst; review is head of data
Data model	catalogue, versions, proposals, approve	architect; approve is head of data
Access	access groups, scopes, memberships, admin log	company admin

Family	Examples	Seat or role
Jobs and events	jobs list, retry, `GET /api/v1/events/stream` (SSE)	data engineer; stream to any signed-in seat, company-filtered
Control	companies, module switches, packs	super admin (off on the public link)

The authoritative list is `sh_platform/route_permissions.yaml` plus the per-module `*.permissions.yaml` files.

9. Testing and quality

Suite	Latest result
UI (vitest)	1755 passed
Backend (pytest)	2144 passed, 1 known timing flake
Production mode (Postgres, Redis)	Two-modes suite: 24 passed, 3 skipped (the 3 need a real Postgres and Redis)

Practice: the agent that writes a package is never its reviewer; packages land through one commit tool; screens are verified by screenshot against the approved artboard; agents are tested live against a quality bar, not only with stubs; a role matrix test checks the permission tables; the 10-use-case verification (`tools/verify_10uc`) walks the synthetic Evergreen universe end to end; unit tests run with the model provider off (no model call). Reviews: the security reviews in section 6, and a design review of each screen.

10. Limits and next steps

Item	Position
DuckDB turn-taking	A DuckDB file takes one writer process at a time, so the web container and the worker share the volume and write in turn. This is why there is one web replica. Planned: route heavy writes through a single writer; evaluate a server warehouse where the client needs concurrent writers
Stores on files	About 350 call sites still use JSON files in production mode (section 5). Planned: move store by store through `StateStore`
Production mode proving	Three production-only tests and the parity checker need a real Postgres and Redis; run them on the target environment before go-live
SORA	Not offered. MAS publishes it through a gateway that needs a free registered key. Planned: add as a provider once the client has a key
Data-level restrictions	Fund (legal entity and portfolio) access is built; column-level and document-type restrictions are planned
Connect an API	A fixed provider list by design; a free-form REST option is not built

Item	Position
Straits Times Index	Licensed; EWS is shown as a labelled stand-in
Postgres connections	Opened per call; a pool is planned
Cloud warehouse push	Needs two named approvers; pilot carries synthetic data only